Stage One Follow-up Package — Revision 1

Oct 14, 2026 · @Shantha

This revision replaces the first follow-up package for review. That package stays unchanged. **Nothing has been executed**: no database statement and no query plan has been sent since the Stage One report. The investigation remains open, and nothing proceeds to installation.

What changed.

- **G1.** The original G1 has no time cap. It is kept as a reference and is not proposed for running. G1c is the same text with a 60-second hint, and has its own checksum. The claim that every statement is capped is corrected: T1, H0 and H1 have no hint, and they read no application table.
- **Hint check.** The check on phpMyAdmin's echo is withdrawn. Section 3 proposes reading MySQL's own timeout counters before and after each capped statement, with the exact statements, their scope and their limits. No lock-release deadline is claimed.
- **E1.** The EXPLAIN FORMAT=TREE plan check, without ANALYZE, is now a fixed step before R20.2r, with its complete SQL. Section 2 sets out the plan findings that allow R20.2r to run and those that stop it. R20.2r's runtime remains untested.
- **G2.** G2 is replaced by G2r, which returns one row per voucher number. Ledger totals are therefore counted once per number even if two voucher rows share a number. Shared numbers are reported apart and are not added into the main reconciliation.
- **R20.2r** gains two columns for zero-amount vouchers. A zero-amount voucher matches almost any voucher-payment row, because every such row has a 0 on one side. Its checksum, and E1's, changed.
- **Review corrections**, from an independent read of the code:
  - the second authoriser, V4, which can post a voucher's lines more than once in one click;
  - R3's top-up voucher, a double-payment path;
  - R1's missing check that the voucher is a transport advance;
  - R4's undo, which resets another session's request;
  - R2's compounding dollar count;
  - the global permission filter;
  - sources for the timings and table engines;
  - the read count, which now includes the by-number lookups;
  - the R20.2 duration;
  - the sign-in-lifetime assumption, withdrawn.

- **Ids.** The candidate causes are now C1–C5, and the aflag reads are L1 and L2, so no id is shared with E1 or A1/A2.

Labels: **[Observed]** means read in code or in Stage One results. **[Deduced]** means reasoning from those. **[Not established]** means not known. **[Proposal]** means for your decision.

1. Tracing the LKR 298,375.00 difference

Two metadata reads (M1, M2), the capped comparison (G1c), a number-level reconciliation (G2r) and two detail reads (G3, G4) would show where the extra credit sits. None of them assumes a cause. That voucher 7134's amount equals the difference is a coincidence to test, not a finding. **[Proposal]**

What is known. [Observed]

- R0's 6,833,545.97 is the sum of credits to 1-6-8-9750 under the numbers of the 38 sup = 0 vouchers. R23.1's 6,535,170.97 is the sum of those vouchers' Amount.
- **V1** (VoucherPaymentController::update, status 1) posts one journal per line: the v_inv rows whose V_tkn equals the number.
  - The credit goes to 1-6-8-9750 when the sup **sent with the request** (:355) is not 1, and only for the four types it handles: Bank Transfer, Cash, Petty Cash, Cheque.
  - Each journal is for the line's Amount, never voucher.Amount.
  - It does not check that the voucher still awaits authorisation.
  - Its rows carry the line's Invoice in Invoice_Num.
- **V4** (ManualUpdateController::hotel_voucher_accept) also authorises.
  - It uses the **stored** sup.
  - Its list holds one entry per voucher number and line Invoice, and for each entry it posts **all** of the voucher's lines. A voucher whose lines carry k different Invoice values is therefore posted k times in one click.
  - Its rows carry Invoice_Num '0'.

Candidate causes and how each would show. None is assumed.

#	Possible cause	Sign in G2r or G3
C1	The voucher's lines differ from its Amount	cr_9750 = line_total ≠ amount: the lines as posted differ from the voucher. cr_9750 ≠ line_total: lines changed after posting, or posted a different number of times (C2)

#	Possible cause	Sign in G2r or G3
C2	Lines posted more than once: V1 run twice, or V4 posting once per distinct line Invoice	journals_9750 = k × line_count and cr_9750 = k × line_total, with k ⩾ 2. cr_9750_invoice_num_0 > 0 points to V4. Repeated legs in G3, told apart by line_invoice
C3	Rows from another source under the same Recpt_No	cr_9750 ≠ cr_9750_voucher_payment_rows; other description types in G3
C4	A number shared with another voucher row (R23.1's 38 numbers are distinct, so the other row would have a different sup or Aflag)	voucher_rows > 1. With journals_9750 = k × line_count, it shows double posting through the shared number
C5	A combination, spread over several vouchers	differences across several G2r rows that add up to 298,375.00

M1. Columns used, with types and defaults.

```
SELECT /*+ MAX_EXECUTION_TIME(60000) */ c.TABLE_NAME, c.COLUMN_NAME,
c.DATA_TYPE, c.COLUMN_TYPE, c.IS_NULLABLE, c.COLUMN_DEFAULT
FROM information_schema.COLUMNS c
WHERE c.TABLE_SCHEMA = 'avenir'
  AND ((c.TABLE_NAME = 'v_inv' AND c.COLUMN_NAME IN ('ID', 'V_tkn',
'Invoice', 'account', 'Amount', 'RCType', 'Date'))
    OR (c.TABLE_NAME IN ('account_runing', 'account_runing_2024') AND
c.COLUMN_NAME IN ('aflag', 'Bank_Date', 'Recpt_No', 'Date', 'Discription',
'Dr', 'Cr', 'ID', 'L_Code'))
    OR (c.TABLE_NAME = 'voucher' AND c.COLUMN_NAME IN ('Vnum', 'Amount',
'W_Date', 'Aflag', 'sup')))
ORDER BY c.TABLE_NAME, c.COLUMN_NAME;
```

M2. Table sizes, engines and indexes.

```
SELECT /*+ MAX_EXECUTION_TIME(60000) */ t.TABLE_NAME, t.ENGINE, t.TABLE_ROWS,
s.INDEX_NAME, s.NON_UNIQUE, s.SEQ_IN_INDEX, s.COLUMN_NAME
FROM information_schema.TABLES t
LEFT JOIN information_schema.STATISTICS s ON s.TABLE_SCHEMA = t.TABLE_SCHEMA
AND s.TABLE_NAME = t.TABLE_NAME
WHERE t.TABLE_SCHEMA = 'avenir'
  AND t.TABLE_NAME IN ('voucher', 'v_inv', 'account_runing',
'account_runing_2024', 'user_acc_tbl')
ORDER BY t.TABLE_NAME, s.INDEX_NAME, s.SEQ_IN_INDEX;
```

G1 (reference only, not proposed for running). This is the section 8 text exactly. Its SHA-256 equals that of the Stage One report's section 8 code block, re-read on 14 October. It has no time cap.

```
SELECT v.Vnum AS voucher_no, v.Amount AS voucher_amount,
       SUM(a.Cr) AS credits_to_9750, COUNT(DISTINCT a.ID) AS journals
FROM avenir.voucher v
JOIN avenir.account_runing a ON a.Recpt_No = v.Vnum AND a.L_Code = '1-6-8-
9750'
WHERE v.sup = 0 AND v.Aflag = 1
GROUP BY v.Vnum, v.Amount
HAVING SUM(a.Cr) <> v.Amount
ORDER BY v.Vnum;
```

G1c (proposed). G1 with a 60-second cap and no other change.

```
SELECT /*+ MAX_EXECUTION_TIME(60000) */ v.Vnum AS voucher_no, v.Amount AS
voucher_amount,
       SUM(a.Cr) AS credits_to_9750, COUNT(DISTINCT a.ID) AS journals
FROM avenir.voucher v
JOIN avenir.account_runing a ON a.Recpt_No = v.Vnum AND a.L_Code = '1-6-8-
9750'
WHERE v.sup = 0 AND v.Aflag = 1
GROUP BY v.Vnum, v.Amount
HAVING SUM(a.Cr) <> v.Amount
ORDER BY v.Vnum;
```

G1 and G1c have five limits:

- They join ledger rows to every voucher row with the number, so a shared number would double their sums.
- They read only account_runing and only credits.
- They drop numbers with no 1-6-8-9750 rows.
- They do not separate rows by source.
- They list only numbers that differ.

G1c is a first look. G2r is the reconciliation.

G2r. Reconciliation at the level of the voucher number.

```
SELECT /*+ MAX_EXECUTION_TIME(60000) */
       n.Vnum AS voucher_no, n.voucher_rows, n.sup0_authorised_rows,
       n.amount_sup0_authorised_rows, n.amount_all_rows,
       COALESCE(li.line_count, 0) AS line_count, COALESCE(li.line_total, 0)
AS line_total,
       COALESCE(c.rows_9750, 0) AS rows_9750, COALESCE(c.journals_9750, 0) AS
```

```
journals_9750,
       COALESCE(c.cr_9750, 0) AS cr_9750, COALESCE(c.dr_9750, 0) AS dr_9750,
       COALESCE(c.vp_cr_9750, 0) AS cr_9750_voucher_payment_rows,
       COALESCE(c.inv0_cr_9750, 0) AS cr_9750_invoice_num_0,
       COALESCE(o.rows_9750_2024, 0) AS rows_9750_in_2024_table
FROM (SELECT w.Vnum, COUNT(*) AS voucher_rows,
             SUM(w.sup = 0 AND w.Aflag = 1) AS sup0_authorised_rows,
             SUM(CASE WHEN w.sup = 0 AND w.Aflag = 1 THEN w.Amount ELSE 0
END) AS amount_sup0_authorised_rows,
             SUM(w.Amount) AS amount_all_rows
      FROM avenir.voucher w
      WHERE w.Vnum IN (SELECT x.Vnum FROM avenir.voucher x WHERE x.sup = 0
AND x.Aflag = 1)
      GROUP BY w.Vnum) n
LEFT JOIN (SELECT i.V_tkn, COUNT(*) AS line_count, SUM(i.Amount) AS
line_total
           FROM avenir.v_inv i GROUP BY i.V_tkn) li ON li.V_tkn = n.Vnum
LEFT JOIN (SELECT a.Recpt_No, COUNT(*) AS rows_9750, COUNT(DISTINCT a.ID) AS
journals_9750,
                  SUM(a.Cr) AS cr_9750, SUM(a.Dr) AS dr_9750,
                  SUM(CASE WHEN UPPER(a.Discription) LIKE 'VOUCHER PAYMENT%'
THEN a.Cr ELSE 0 END) AS vp_cr_9750,
                  SUM(CASE WHEN a.Invoice_Num = '0' THEN a.Cr ELSE 0 END) AS
inv0_cr_9750
           FROM avenir.account_runing a WHERE a.L_Code = '1-6-8-9750' GROUP
BY a.Recpt_No) c ON c.Recpt_No = n.Vnum
LEFT JOIN (SELECT b.Recpt_No, COUNT(*) AS rows_9750_2024
           FROM avenir.account_runing_2024 b WHERE b.L_Code = '1-6-8-9750'
GROUP BY b.Recpt_No) o ON o.Recpt_No = n.Vnum
ORDER BY n.Vnum;
```

What G2r prevents, and what it does not. [Deduced from the statement text]

- Every source is summed on its own before any join: voucher rows, lines, credits to 1-6-8-9750 in each ledger table. One source row therefore cannot be multiplied by another source's rows.
- That alone would not stop a shared number repeating the same ledger totals on two voucher rows. G2 had that weakness, because it returned one row per voucher row.
- G2r returns **one row per voucher number**. The voucher side is grouped by number first, and the ledger and line totals are joined to that single row. Adding cr_9750 or line_total down the result therefore counts each number's ledger once, however many voucher rows share it.

How a shared number is handled.

- A row with voucher_rows > 1 is a shared number. The other voucher row has a different sup or Aflag, since R23.1's numbers are distinct.
- For such a row, amount_all_rows and amount_sup0_authorised_rows show both voucher sides. The ledger totals cannot be split between the voucher rows from these columns.
- Shared numbers are therefore reported as a separate group, with their figures shown in full. They are **excluded from the main voucher-to-ledger comparison**, not divided or estimated.

Built-in checks.

- Expected rows: 38, one per number.
- SUM(amount_sup0_authorised_rows) must equal 6,535,170.97 (R23.1). A difference stops the run.
- SUM(cr_9750) either reproduces R0's 6,833,545.97 or is reported as not reproduced.

G3. Every leg of every journal that credits 1-6-8-9750 under the 38 numbers.

```
SELECT /*+ MAX_EXECUTION_TIME(60000) */
       a.Recpt_No AS voucher_no, a.ID AS journal_id, a.L_Code, a.Dr, a.Cr,
a.Date, a.Dt AS entered_at,
       CASE WHEN UPPER(a.Discription) LIKE 'VOUCHER PAYMENT%' THEN 'voucher
payment'
            WHEN a.Discription LIKE 'Hotel Reservation%' THEN 'hotel
reservation accrual'
            WHEN a.Discription LIKE 'Lunch or Other Fee%' THEN 'lunch or
other fee'
            WHEN a.Discription LIKE 'Brought Forward Balance%' THEN 'brought
forward'
            WHEN a.Discription IS NULL OR a.Discription = '' THEN '(empty)'
            ELSE 'other (text withheld)' END AS description_type,
       a.Invoice_Num AS line_invoice, a.aflag, a.Bank_Date,
       (SELECT GROUP_CONCAT(u.id ORDER BY u.id) FROM avenir.user_acc_tbl u
WHERE u.user = a.Cashier) AS entering_login_ids
FROM avenir.account_runing a
WHERE a.ID IN (SELECT b.ID FROM avenir.account_runing b
               WHERE b.L_Code = '1-6-8-9750'
                 AND b.Recpt_No IN (SELECT v.Vnum FROM avenir.voucher v WHERE
v.sup = 0 AND v.Aflag = 1))
ORDER BY a.Recpt_No, a.ID, a.Dr DESC, a.L_Code;
```

line_invoice tells the authorisers apart: '0' for V4, the line's Invoice (normally the tour) for V1. A '0' can also come from a line whose own Invoice is '0', so it is an indicator, not proof.

G4. The 38 vouchers' lines. Description and Payee are left out because they can hold names.

```
SELECT /*+ MAX_EXECUTION_TIME(60000) */ i.V_tkn AS voucher_no, i.ID AS
line_id, i.Invoice AS line_invoice, i.account, i.Amount, i.RCType, i.Date
FROM avenir.v_inv i
WHERE i.V_tkn IN (SELECT v.Vnum FROM avenir.voucher v WHERE v.sup = 0 AND
v.Aflag = 1)
ORDER BY i.V_tkn, i.ID;
```

Scope and limits.

- **Scope:** schema `avenir`, tables voucher, v_inv, account_runing, account_runing_2024 and user_acc_tbl, which supplies ids only. M1 and M2 read information_schema filtered to `avenir`.
- **Size:** R0 counted 125 rows on 1-6-8-9750 in all, so G3 should return well under 300 rows.
- **History:** the reads show the present state only. Which screen wrote a row is not recorded. Legacy screens keep no audit of amount or line edits, and tpd_audit starts on 11 October. Earlier deletions are not visible.
- **Journal-number collisions** (SX-7) would make G3 return legs under other numbers; they would show by voucher_no.
- **V_tkn:** a 6 October schema dump gives V_tkn as bigint, which makes the joins numeric. M1 confirms this on the target. If it were text, spacing differences would still match numerically, and the per-number grouping could split. That would be reported.
- **Result:** the reads can show where the extra LKR 298,375.00 sits, not why. Any correction is the accountant's decision.

2. R20.2r and the plan check E1

R20.2r would answer R20.2's question with the same coverage, but it runs **only if E1's plan shows no per-voucher pass over a ledger table**. Its runtime is **untested**. The original 36-minute statement is not proposed again. **[Proposal]**

What R20.2 asked. R20.2 takes each authorised voucher found by number in neither ledger table (3,921, as R20.1 implies). It counts how many have a "VOUCHER PAYMENT" row with the same date and with Dr or Cr equal to the voucher amount, in either table, grouped by voucher month.

Why it took 36 minutes. [Deduced] The plan was never read.

- For each of the 3,921 vouchers, the OR of two correlated EXISTS can mean a pass over both ledger tables, with UPPER() worked out per row.

- R0 counted 11,364 rows in account_runing and 15,900 in account_runing_2024, so that is up to about 107 million row tests.
- The by-number part of the same filter ran in 0.393 seconds as R20.1. This is phpMyAdmin's "Query took" figure from Stage One, kept in this session's capture but not in the result files.

R20.2r.

```
SELECT /*+ MAX_EXECUTION_TIME(120000) */
       DATE_FORMAT(v.W_Date, '%Y-%m') AS voucher_month,
       COUNT(*) AS not_found_by_number,
       SUM(cur.d IS NOT NULL OR old.d IS NOT NULL) AS
candidate_same_date_and_amount,
       SUM(cur.d IS NOT NULL AND old.d IS NULL) AS
candidate_current_table_only,
       SUM(cur.d IS NULL AND old.d IS NOT NULL) AS candidate_2024_table_only,
       SUM(cur.d IS NOT NULL AND old.d IS NOT NULL) AS candidate_both_tables,
       SUM(v.Amount = 0) AS zero_amount_vouchers,
       SUM(v.Amount = 0 AND (cur.d IS NOT NULL OR old.d IS NOT NULL)) AS
zero_amount_candidates
FROM avenir.voucher v
LEFT JOIN (SELECT a.Date AS d, a.Dr AS amt FROM avenir.account_runing a WHERE
UPPER(a.Discription) LIKE 'VOUCHER PAYMENT%'
           UNION
           SELECT a.Date, a.Cr FROM avenir.account_runing a WHERE
UPPER(a.Discription) LIKE 'VOUCHER PAYMENT%') cur
       ON cur.d = DATE(v.W_Date) AND cur.amt = v.Amount
LEFT JOIN (SELECT b.Date AS d, b.Dr AS amt FROM avenir.account_runing_2024 b
WHERE UPPER(b.Discription) LIKE 'VOUCHER PAYMENT%'
           UNION
           SELECT b.Date, b.Cr FROM avenir.account_runing_2024 b WHERE
UPPER(b.Discription) LIKE 'VOUCHER PAYMENT%') old
       ON old.d = DATE(v.W_Date) AND old.amt = v.Amount
WHERE v.Aflag = 1
  AND NOT EXISTS (SELECT 1 FROM avenir.account_runing a WHERE a.Recpt_No =
v.Vnum AND UPPER(a.Discription) LIKE 'VOUCHER PAYMENT%')
  AND NOT EXISTS (SELECT 1 FROM avenir.account_runing_2024 b WHERE b.Recpt_No
= v.Vnum AND UPPER(b.Discription) LIKE 'VOUCHER PAYMENT%')
GROUP BY voucher_month
ORDER BY voucher_month;
```

Same coverage. [Deduced from the statement texts]

- The WHERE clause is R20.2's word for word.

- A row matches in R20.2 when its date equals the voucher date and its Dr or its Cr equals the amount. That is exactly when (date, Dr) or (date, Cr) equals (voucher date, amount).
- Each derived table holds those pairs from every “VOUCHER PAYMENT” row of one ledger table. NULLs match nothing in either version.
- The columns as far as candidate_both_tables answer R20.2′s question. The last two are additions.

No double counting. UNION (distinct) keeps each pair once in each derived table. Each voucher therefore meets at most one row from each table, and COUNT(*) counts voucher rows exactly as R20.2 did. This holds because Date is a date and Dr and Cr share one decimal type, as the 6 October dump shows; M1 confirms it on the target.

Zero amounts. Every voucher-payment journal has a 0 on one side. A zero-amount voucher therefore matches on almost any date that has such a row. R20.2 counted these as candidates too. The two new columns let them be read apart, and they are not removed.

Built-in checks.

- not_found_by_number for each month must equal R20.1′s not_in_current_ledger minus found_by_number_in_2024: 3,921 in all.
- The three “only or both” columns must add up to candidate_same_date_and_amount for each month.
- Either failure stops the run.

Reads.

- Building the two pair sets reads each ledger table twice: 2 × (11,364 + 15,900), or about 54,500 rows.
- On top of that come the by-number lookups into both tables for each of the 7,654 authorised vouchers that R0 counted, as in R20.1.
- **Runtime: untested.** No timing for R20.2r exists, and none is claimed.

E1. The plan check (EXPLAIN FORMAT=TREE, without ANALYZE).

```
EXPLAIN FORMAT=TREE SELECT /*+ MAX_EXECUTION_TIME(120000) */
       DATE_FORMAT(v.W_Date, '%Y-%m') AS voucher_month,
       COUNT(*) AS not_found_by_number,
       SUM(cur.d IS NOT NULL OR old.d IS NOT NULL) AS
candidate_same_date_and_amount,
       SUM(cur.d IS NOT NULL AND old.d IS NULL) AS
candidate_current_table_only,
       SUM(cur.d IS NULL AND old.d IS NOT NULL) AS candidate_2024_table_only,
       SUM(cur.d IS NOT NULL AND old.d IS NOT NULL) AS candidate_both_tables,
```

```
       SUM(v.Amount = 0) AS zero_amount_vouchers,
       SUM(v.Amount = 0 AND (cur.d IS NOT NULL OR old.d IS NOT NULL)) AS
zero_amount_candidates
FROM avenir.voucher v
LEFT JOIN (SELECT a.Date AS d, a.Dr AS amt FROM avenir.account_runing a WHERE
UPPER(a.Discription) LIKE 'VOUCHER PAYMENT%'
           UNION
           SELECT a.Date, a.Cr FROM avenir.account_runing a WHERE
UPPER(a.Discription) LIKE 'VOUCHER PAYMENT%') cur
       ON cur.d = DATE(v.W_Date) AND cur.amt = v.Amount
LEFT JOIN (SELECT b.Date AS d, b.Dr AS amt FROM avenir.account_runing_2024 b
WHERE UPPER(b.Discription) LIKE 'VOUCHER PAYMENT%'
           UNION
           SELECT b.Date, b.Cr FROM avenir.account_runing_2024 b WHERE
UPPER(b.Discription) LIKE 'VOUCHER PAYMENT%') old
       ON old.d = DATE(v.W_Date) AND old.amt = v.Amount
WHERE v.Aflag = 1
  AND NOT EXISTS (SELECT 1 FROM avenir.account_runing a WHERE a.Recpt_No =
v.Vnum AND UPPER(a.Discription) LIKE 'VOUCHER PAYMENT%')
  AND NOT EXISTS (SELECT 1 FROM avenir.account_runing_2024 b WHERE b.Recpt_No
= v.Vnum AND UPPER(b.Discription) LIKE 'VOUCHER PAYMENT%')
GROUP BY voucher_month
ORDER BY voucher_month;
```

What E1 does and does not do.

- Without ANALYZE, MySQL builds and prints the plan but does not run the query.
- It still opens the tables, takes brief metadata locks, and may read index statistics.
- R20.2r has no constant subquery that the optimiser would evaluate while planning **[Deduced]**.
- E1′s own effect on the timeout counters is not established. Its H1 reading is recorded but not used as a test.

Findings that allow R20.2r. All must hold:

1. E1 returns a plan with no error and no warning.
2. Each of the two UNION sets appears once, as a union materialisation, outside any per-voucher loop. It is not marked as rebuilt for each row (“invalidate on row”) and not inside a dependent subquery.
3. voucher reaches each set by a hash join, or by an index lookup on the materialised set (for example “Index lookup on cur using <auto_key…>”).
4. Each NOT EXISTS is an antijoin or a dependent lookup that uses an index on Recpt_No, or a hash antijoin.

5. No ledger table (account_runing or account_runing_2024) is scanned inside a loop over vouchers.

Findings that stop the run. Any one is enough:

- an error or warning;
- a full scan of a ledger table under a dependent subquery or inside the voucher loop;
- a set rebuilt per row, or scanned per voucher instead of looked up;
- a plan that cannot be read with confidence against points 1–5.

I would then report the plan as captured. R20.2r would not run, and any rewording would need your approval of new text.

Even with a good plan, R20.2r runs only if the timeout checks in section 3 confirmed that MySQL applied the hint on the earlier capped statements. Its runtime stays a measured result, not a promise.

3. Timeout checks: how the caps would be verified

Whether MySQL applied each MAX_EXECUTION_TIME hint would be checked from the server's own timeout counters, read before and after every capped statement. phpMyAdmin's echo of the query text is not used as evidence. Even a confirmed cap is not a lock-release deadline. **[Proposal]**

Why phpMyAdmin's echo is not enough. A hint that appears unchanged in phpMyAdmin's response shows only that phpMyAdmin sent it. MySQL ignores a hint it cannot apply. Whether phpMyAdmin passes on any warning MySQL raises for a SELECT is not established.

The server's counters. MySQL keeps three server-wide status counters for this. Their meaning, as documented by MySQL:

- **Max_execution_time_set:** SELECT statements for which a nonzero timeout was set, either by the hint or by the max_execution_time variable.
- **Max_execution_time_set_failed:** SELECT statements for which setting the timeout failed.
- **Max_execution_time_exceeded:** SELECT statements stopped because they ran over.

On this server they have not yet been read.

H0. Settings, read once at the start.

```
SELECT @@GLOBAL.max_execution_time AS global_max_execution_time,
       @@SESSION.max_execution_time AS session_max_execution_time,
       @@GLOBAL.performance_schema AS performance_schema_on,
       @@version AS mysql_version;
```

- **Scope:** four server variables, no table.
- **Expected:** both max_execution_time values 0, which is MySQL's default; performance_schema 1; version 8.4.7.
- **Stop if** either value is not 0. Every SELECT would then get a timeout, and the counters could not show whether a hint was applied. Also stop if performance_schema is 0, because H1 would then have nothing to read.

H1. The three counters, read before the first capped statement and after each one.

```
SELECT VARIABLE_NAME, VARIABLE_VALUE
FROM performance_schema.global_status
WHERE VARIABLE_NAME IN ('Max_execution_time_set',
'Max_execution_time_set_failed', 'Max_execution_time_exceeded')
ORDER BY VARIABLE_NAME;
```

- **Scope:** three rows of performance_schema.global_status. This is a server status view **outside schema `avenir`**. It holds counters only, no application data, but it does need your explicit approval because it is outside `avenir`.
- **Expected after each capped statement:** Max_execution_time_set up by exactly 1, Max_execution_time_set_failed unchanged, and Max_execution_time_exceeded unchanged. The exception is a statement that ended with error 3024, where exceeded rises by 1.
- **Stop if** set does not rise by 1, set_failed rises, or exceeded rises without a 3024 error. The cap is then not confirmed, and no further capped statement runs.
- **Interference:** the counters are server-wide, so another session's timed SELECT would also move them. The application code has no MAX_EXECUTION_TIME or max_execution_time in app/ **[Observed: code copy]**. With the local app idle, a movement other than our own is not expected. Any unexplained movement is a stop.

What a confirmed cap does and does not give.

- **Confirmed:** that MySQL set a timeout for that SELECT.
- **Not given: a release time.** MySQL stops an over-running SELECT when its execution next checks for interruption. The delay after the limit is not specified, and none is claimed.
- **Not given: lock waits.** Whether time spent waiting for a MyISAM table lock counts towards the limit is not established.

- **Locks and the app:** a stopped or finished statement releases its MyISAM read locks when it ends. App writes queued behind it then go first, and MyISAM makes later reads wait for those writes. Delays in the app can therefore outlast the statement **[Deduced]**.
- **Where the locks apply:** the eight payment tables, including voucher, v_inv and account_runing, are MyISAM according to R0 (R10, “MyISAM tables”) and r3.3. account_runing_2024′s engine is not on record; M2 shows it.
- **phpMyAdmin and PHP time limits** are separate and their values on this install are not established. They can end the web request while MySQL carries on, or the reverse. A lost response is reported as a failure.
- **E1:** the counters are not used to judge EXPLAIN, which does not run the query.
- **Uncapped statements:** T1, H0 and H1 read no application table. G1 (reference) is not run.

4. The ledger's aflag and Bank_Date (L1, L2)

Neither field marks a row as void in the application code. On code evidence, neither changes which rows count for R20–R23 or for balances. What the data holds in them is not established, and L1 and L2 would show it. Apart from the renamed ids, this section is unchanged from the first package.

Verified code behaviour. [Observed: code copy; backup copies of controllers excluded]

Field	Written by	Read by
aflag = 3	V1 (through acc_array, including the sup = 2 clearing journal); V4; V5 (:476, :497); JournalEntryController (:113, :248, :276)	PettyCashReportController (:50, :101): the petty-cash report lists only aflag 3 rows
aflag = 1	DriverFundReturnController (:161, :183); TourDetailsSearchController advance_delete: “takes a tour advance out of the advance list” by setting every leg from 0 to 1, audited	TourDetailsSearchController: the advance list shows rows on 6-11-107-10381 with aflag 0, and advance_delete accepts only entries whose legs are all 0
aflag copied	ManualUpdateController :686 (bank_recon_record_insert, a retired tool that now refuses)	—
aflag not set	most other posters, for example expense acceptance (R3) and hotel accruals	—

Field	Written by	Read by
Bank_Date	listed as an allowed field in AccountRunningModel and AccountRunning2024Model; **no code writes it**	**no code reads it**

voucher.bank_date is a different column, on the voucher table.

What follows. [Deduced]

- aflag records which kind of routine posted a row. For tour advances it records whether the entry is still in the advance list.
- aflag 1 on an advance does not remove its rows from the ledger. Every reader other than the advance list still counts them.
- Any Bank_Date values came from outside the current code.

Open points.

- **[Not established]** whether any row carries an aflag value written outside these paths.
- The column default (M1).
- Whether an advance taken out of the list should be treated as reversed is a question for the accountant.

L1. How aflag and Bank_Date are spread, by ledger table and description category.

```
SELECT /*+ MAX_EXECUTION_TIME(60000) */ 'account_runing' AS ledger_table,
a.aflag,
       CASE WHEN UPPER(a.Discription) LIKE 'VOUCHER PAYMENT%' THEN 'voucher
payment'
            WHEN a.Discription LIKE 'Hotel Reservation%' THEN 'hotel
reservation accrual'
            WHEN a.Discription LIKE 'Lunch or Other Fee%' THEN 'lunch or
other fee'
            WHEN a.Discription LIKE 'Brought Forward Balance%' THEN 'brought
forward'
            WHEN a.Discription IS NULL OR a.Discription = '' THEN '(empty)'
            ELSE 'other (text withheld)' END AS description_type,
       COUNT(*) AS row_count, SUM(a.Dr) AS dr_total, SUM(a.Cr) AS cr_total,
       SUM(a.Bank_Date IS NOT NULL) AS rows_with_bank_date, MIN(a.Bank_Date)
AS first_bank_date, MAX(a.Bank_Date) AS last_bank_date
FROM avenir.account_runing a
GROUP BY a.aflag, description_type
UNION ALL
SELECT 'account_runing_2024', b.aflag,
       CASE WHEN UPPER(b.Discription) LIKE 'VOUCHER PAYMENT%' THEN 'voucher
```

```
payment'
            WHEN b.Discription LIKE 'Hotel Reservation%' THEN 'hotel
reservation accrual'
            WHEN b.Discription LIKE 'Lunch or Other Fee%' THEN 'lunch or
other fee'
            WHEN b.Discription LIKE 'Brought Forward Balance%' THEN 'brought
forward'
            WHEN b.Discription IS NULL OR b.Discription = '' THEN '(empty)'
            ELSE 'other (text withheld)' END AS description_type,
       COUNT(*), SUM(b.Dr), SUM(b.Cr),
       SUM(b.Bank_Date IS NOT NULL), MIN(b.Bank_Date), MAX(b.Bank_Date)
FROM avenir.account_runing_2024 b
GROUP BY b.aflag, description_type
ORDER BY 1, 2, 3;
```

L2. aflag and Bank_Date on the Stage One rows. Expected 41 rows: 11 from R21 and 30 from R22. G3 covers the 38 sup = 0 vouchers.

```
SELECT /*+ MAX_EXECUTION_TIME(60000) */ a.ID AS journal_id, a.L_Code, a.Dr,
a.Cr, a.Date, a.aflag, a.Bank_Date
FROM avenir.account_runing a
WHERE a.ID IN (24654, 24734, 25804, 25915, 25918, 27724, 27725, 28358)
   OR a.Date > '2026-12-31'
ORDER BY a.ID, a.Dr DESC, a.L_Code;
```

How the results would be read. “Voucher payment” rows with aflag 3 and no Bank_Date match the code. Anything else is reported as found. Unusual values on Stage One rows are flagged for review; no row is dropped from any count.

5. Containment for the six additional routes, corrected

The recommendation is unchanged: refuse R1 and R2, and guard R3–R6 with the A2 locks plus the desk’s voucher-number lock. The analysis is corrected in nine places, marked **(corrected)**. Files are the code copy where its checksum equals the target’s, and the staged target files for the desk and its libraries. **[Proposal]**

Route (POST)	Main risk	Proposed safeguard	Effect on staff
R1 dashboard2/transport/driver_advance/delete_advance	deletes any pending voucher by number, even while it is being authorised	**refuse**; reject through Bank Payment Check instead	transport staff ask Accounts to reject a wrong advance
R2 tour/reserve_hotel/hotel_payment	double payment; nights left at flag 10 with no voucher; dollar count inflated	**refuse**; hotel advances through payment requests	reservations staff raise a request
R3 tour/tour_accounts/accept_approved_expenses	journals posted twice; top-up voucher pays settled expenses again	**guard** (tour lock, claim each expense, fix the top-up rule)	a second click is refused
R4 tour/tour_payment_dashboard/bulk_create	voucher for a request cancelled meanwhile; undo resets another session's request	**guard** (request locks, status re-read, narrower undo)	"try again" when a request is in use
R5 tour/tour_payment_dashboard/initiate	bank change silently skipped after authorisation	**guard** (AUTH lock)	may wait up to 10 seconds
R6 tour/tour_payment_dashboard/bulk_initiate	as R5, for a batch	**guard** (AUTH held for the run)	other authorisations wait

Common to all six. [Observed] unless marked.

- **X1. Voucher numbers come from two kinds of writer.**
  - Vouchers::issue (R4) takes the named lock `tpd_voucher_issue` and checks the numbers are free.
  - InvoiceNumberModel::incrementVoucherToken (R2, R3 and older screens) reads the counter and writes it back with no lock.

  - Two writers can take the same number **[Deduced]**.
  - Setup adds a unique index on voucher.Vnum only if no duplicates existed. M2 shows whether the target has it.
- **X2.** V1 marks a voucher with no lines as authorised and posts nothing.
- **X3.** Lines are tied to a voucher by number only. An orphan line is posted with whichever voucher holds its number **[Deduced]**.
- **X4 (corrected).**
  - Two global filters apply to all six: UserSessionFilter (sign-in) and UserPermissionFilter (Filters.php :46, :48, with exceptions). What UserPermissionFilter checks for these routes needs a read (F-7).
  - None of the six is in the E8 registry, which covers accounts/* only.
  - The global CSRF filter is commented out.
- **X5.** A2′s AUTH and REQ locks bind only code that asks for them.
- **X6 (new). V4 also authorises.**
  - It uses the stored sup and posts all of a voucher′s lines once per distinct line Invoice.
  - It never moves hotel nights from flag 10 to 1, unlike V1.
  - C0.15 already proposes refusing V4.

R1. delete_advance (DriverAdvancePaymentController :383)

- **Reached from:** transport_advance.php:784. **Internal calls:** none.
- **Existing checks:** a voucher row with that number exists; that row′s Aflag is 0; the tour posted by the page is not cancelled.
- **(corrected)** Nothing checks that the voucher is a transport advance, or that it belongs to the posted tour. Any pending voucher number can be deleted, and the cancel check and marker reset use the tour id the browser sends.
- **Writes:** deletes every voucher row and every line with that number, then sets the posted tour′s transport_advance marker to 0.
- **Conflicts [Deduced]:**
  - V1 can authorise between the check and the delete, leaving a journal with no voucher.
  - Only the first row is checked, but all rows under the number go.
  - The marker is reset even if another advance remains.
- **Partial failure:** orphan lines (X3), or a marker left at 1.
- **Safeguard: refuse** (recommended); the advance is rejected in Bank Payment Check, which keeps the record. The alternative guard is in the first package, plus a check that the voucher is a transport advance of the posted tour.

- **Staff:** Accounts rejects a wrong advance. Rejection does not reset the marker; whether that blocks a new advance needs F-3.

R2. hotel_payment (HotelManagementController :480)

- **Reached from:** booking_details.php:2477; the forecast page's call is commented out. **Internal calls:** none.
- **Existing checks:** the tour is not cancelled; required fields are filled; the amount is a number of 0 or more; every night is confirmed; it refuses only if a night has flag 1 and its dollars reach the total.
- **Writes, in order:**
  1. Each night gets flag 10 and more dollars.
  2. The counter is taken, with no lock.
  3. The line is written **before** the voucher.
  4. The voucher is written: sup 1, Aflag 0.
- **(corrected) Dollar count.** The paid-dollar count compounds from night to night (:571): each night is given the running total of the nights before it plus the payment. Over several nights it is overstated **[Observed: code; effect Deduced]**.
- **(corrected) Every hotel on those dates.** V1 approval (:547) and rejection (:603) update hotel_payment_flag for every hotel on those dates of the tour, not only this one.
- **Conflicts [Deduced]:**
  - Flag 10 is not checked, so a second submit creates a second voucher.
  - Payment requests for the same nights are not checked.
  - Flag 10 blocks booking cancellation (B1). V4 never clears it (X6).
- **Partial failure:** nights at 10 with an orphan line and no voucher to reject.
- **Safeguard: refuse** (recommended, consistent with A2's refusal of save_payment).
- **Staff:** hotel advances are raised as payment requests.

R3. accept_approved_expenses (TourAccountsController :2239)

- **Reached from:** approved_expenses_report.php:521; payment_approval_v2.php:1476. **Internal calls:** none.
- **Existing checks:** the r1b approver role; tour and bank given; at least one "approved" expense; log-sheet mileage for every allocated driver; a payable above 0. There is no own-request check.
- **(corrected) Settled drivers.** When a settlement voucher already exists for the driver and tour (:2388–2393), mileage and advances are set to 0. The payable then equals **all** "posted" expenses of that driver and tour, including ones already paid in the earlier

settlement. A top-up voucher for that sum is raised. This is a double-payment path **[Observed: code :2388–2435; effect Deduced]**.

- **Writes:** for each expense, a journal (Dr supplier / Cr driver), then the expense is marked “posted”. For each driver, a voucher number with no lock, a voucher, then its line.
- **Conflicts [Deduced]:**
  - Two calls post every journal twice and can raise two vouchers.
  - A rejection can run at the same time.
  - Advances changing (V5, V1, R1) after the amount is worked out make it wrong.
- **Partial failure:** a re-post on retry, or a voucher without a line (X2).
- **Safeguard:** guard.
  - Lock the tour.
  - Claim each expense with a conditional status change before posting it.
  - Raise a top-up only for expenses posted in this call.
  - Take the voucher-number lock, and check every write.
  - An own-request check is your separation decision.
- **Staff:** a second click is refused; settlements otherwise unchanged.

R4. bulk_create (desk :146, target file)

- **Reached from:** tpd_payables.php:328. **Internal calls:** no controllers; Bulk::create, Batches, Vouchers::issue, SupplierBank::verify.
- **Existing checks:**
  - privilege; Setup version 7; paying account; verified supplier bank or a verifier's confirmation; separation; a one-time token;
  - eligibility at status 4 or 6;
  - inside the issue: the `tpd_voucher_issue` lock, a check that the numbers are free, a conditional voucher_created change, and a count before finishing.
- **Conflicts [Deduced]:** status is not re-read under a lock, so a request cancelled meanwhile can still get a voucher. Writers outside the number lock can take a number between the check and the write.
- **Partial failure (corrected).**
  - On MyISAM the undo is by hand. It leaves the rows of the voucher being written when the failure happened, and it sets the counter back even if another writer moved it.

    - It also sets voucher_created = 0 on **every** request in the run (Bulk.php:246). That includes one that another session vouchered meanwhile, which could then be vouchered a second time.
- **Safeguard:** guard.
    - Take REQ locks in id order and re-read status 4 or 6.
    - Make the undo cover only this run's own vouchers and requests, including the in-flight one.
    - Set the counter back only if it has not moved.
- **Staff:** "try again" when a request is in use.

R5. initiate (desk :1176, target file)

- **Reached from:** desk pages (F-1), and from R6 at :283. **Internal calls:** none.
- **Existing checks:**
    - the accountant privilege;
    - not cancelled, rejected or finalized; no earlier initiation, with a unique key on the voucher;
    - a valid account, with a reason if it changes;
    - **(corrected)** only bulk-child vouchers (with a parent batch) must come through R6;
    - separation; a date not in the future; the amount matches.
- **Writes:** batch vouchers are re-rated (lines, then Amount where Aflag is 0). Then the initiation row is written, and the voucher's bank is changed only `WHERE Aflag = 0`.
- **Conflicts [Deduced]:** if V1 authorises first, the bank change silently does nothing. The re-rating checks Aflag once, then changes lines.
- **Safeguard:** guard. Take AUTH, re-read Aflag, refuse a needed bank change on a voucher no longer pending, and check every write.
- **Staff:** may wait up to 10 seconds.

R6. bulk_initiate (desk :228, target file)

- **Reached from:** tpd_bulk.php:158. **Internal call:** R5 per issued voucher (:283).
- **Existing checks:** the accountant privilege; the lock `tpd_bulk_<id>` (20 s); resumable; stops at the first error.
- **(corrected)** The batch detail is read at :235, **before** the lock is taken at :240. The pending list it uses can therefore be stale.
- **Safeguard:** guard. Take AUTH once for the run, then the batch lock, then re-read the batch detail.
- **Staff:** other authorisations wait during a bulk initiation.

The gap in Stage One section 3 (corrected)

Stage One section 3 did not list all routes that **raise pending vouchers**. F5 shows more writers of Aflag 0. Their routes, from the route file, are below. **[Observed]**

Route (POST)	Method
accounts/voucher_payments/save	VoucherPaymentController::save (manual voucher entry)
accounts/creditor_invoice/complete_payment	CreditorInvoiceController::complete_payment
dashboard2/transport/driver_advance/tour_transport	DriverAdvancePaymentController::tour_transport
dashboard2/transport/driver_settlement/setteled_payment	DriverPaymentSettlementController::complete_payment
dashboard2/hotel_management/hotel_payment/hotel_pay	NewDashboard/hotel/HotelPaymentController::hotel_payment
tour/booking_confirm/tour_transport and /setteled_payment	BookingConfirmController::tour_transport, ::complete_payment
not found by this search	UtilityBillPaymentController::createUtilityVoucher (private); NewCreditorInvoiceController::complete_payment123

None of these authorises a voucher itself. **[Deduced, corrected]** Their vouchers are authorised by V1, by V4 when a line's description starts "Hotel Reservation", or by the desk through V1. C0.15 would govern all three. They share X1 and X3, and hotel_pay is a second hotel-advance route like R2.

Further reads needed before approval

Code reads in the linked folder, read-only:

- **F-1** The pages behind R1–R6.
- **F-2** DriverAdvancePaymentController::tour_transport and transfer_amount.
- **F-3** Every reader of consignee.transport_advance.
- **F-4** Batches::prepare and finish, PayDesk::isCancelled, Separation::check, Perms and its groups; reject_approved_expenses and the other writers of driver_expenses status.
- **F-5** Checksums against R0's manifest for AccessPolicyRoutes.php, Perms.php, Batches.php, InvoiceNumberModel.php and VoucherPaymentModel.php.
- **F-6** The routes in the gap table, read as fully as R1–R6.
- **F-7** UserPermissionFilter: what it checks for each of the six routes.

Database read: M2.

6. Evidence inventory and durable location

The Stage One evidence sits in four temporary places. I propose one evidence folder outside the web server's folders, filled only after you approve it. The inventory is unchanged from the first package except for the three corrections marked below. **[Observed]** for the inventory; **[Proposal]** for the location.

Result files (this session's workspace, stage1/). Full SHA-256:

File	Content	SHA-256
R20.1.txt	26 rows	398d0a71822352b45dfaa6c26c4ab1cc6e5057ff2c282bacc4241cd4fcb699b1
R20.2.txt	a note only: no result was captured	25b6b0b0468814e3ff7df4dd002e7d00d5765935e4f77e58775f9b74332eafbc
R20.3.txt	1 row (0)	de37b37de45d114784567b0c9eaadf22a09f58c7cda4eb2f6bab55e6a69e05f4
R21.1.txt	8 rows	4379d59e5fcca30c67b29382174888f8bb83f03fbd3183f804bcc81ed9677d4d
R21.2.txt	11 rows	0c2af3edf99f106f6e73c18b692d90adea614225e40242af2f511e59d5baa79e
R22.1.txt	30 rows; free text in one column summarised	76e90f2be59ce98506dbbc6243e0671052808978cd57beb9611781e0b24fbbe7
R22.2.txt	30 rows, same treatment	bcec4b4bc43089f1d6db651772d6ef647395ac10f4b74bd88bcc4e192811a7cf
R23.1.txt	38 rows	27753a65f1e8e49f995b413310dde2341caf13609b8efc93945eef5559e8eec0
R23.2.txt	1 row	8f40d47f38876a67ab135c7a0d857a8909e95ab49b4774051f79b2e4d5fc3317
README.txt	what the files are	87c9661089b4f461ba95377738f01c889a3d658efb31230e35e03f87f625fbf7

Limits.

- The files were written from parsed results, not from raw responses.

- **(corrected)** phpMyAdmin's "Query took" times, such as 0.393 seconds for R20.1, are in this session's capture log but not in the files. T1–T3 values are in the Stage One report.

Code-check outputs (the linked shell's scratch area, outside your folders):

File	Content	SHA-256 (start)
f3.txt	F3, 136 lines	b750afc81f8c3c8b
f4.txt	F4, 104 lines	1ceefc242ea50284
f7.txt	F7, 216 lines	8b919ff0e9ef7bd8
f9.txt	F9, 46 lines	3b550ea62ef8c452
f11_before.txt, f11_after.txt	44 checksums each, identical	3ff4aaaad4e79552
r30_vs_r0.txt	empty: all files matched R0's manifest	e3b0c44298fc1c14
r0/final_files.sha256	R0's manifest, 2,487 files	4983b936e77ac759

The outputs of F0, F1, F2, F5, F6, F8 and F10 were printed but not saved. They can be re-saved word for word from the session output.

Staged copies (this session's uploads area): the target's TourPaymentDeskController.php (6a72efb8b8071ee9), Bulk.php (973feabc144732e6) and Vouchers.php (68eee3259bd54fc4), staged at 20:27 UTC. Older copies in the same area are not Stage One evidence.

The uncaptured R20.2 response.

Item	Value
Sent	20:32:25.514 UTC
Returned	21:08:26.292 UTC
Duration **(corrected)**	2,160,766 ms by the page timer, from just before the request to the parsed result. The two timestamps are 2,160,778 ms apart, because they were taken at slightly different points.
Response	HTTP 200; phpMyAdmin reported success; no result table and no message were found
Raw body	not kept: my capture discarded it, which was my omission

Item	Value
Network log	the browser pane still held it when read while the first package was being prepared (request 14036.8129). Only its first part could be read: the usual page-settings block, with a phpMyAdmin session token. Whether the log still exists now is not established

(corrected) The sign-in-lifetime explanation is withdrawn. Statements sent at 20:59–21:01 in the same phpMyAdmin session succeeded. The cause of the missing result is not established.

Proposed durable location.

- **Where:** a new folder outside C:\wamp64\www, for example C:\AvenirEvidence\2026-10-14-stage1, or one you name. Anything under www can be served by Apache.
- **What goes in:**
  - the result files and README;
  - the code-check outputs and the R0 manifest;
  - the re-saved F0–F10 outputs;
  - every statement text with its SHA-256;
  - PDF exports of the Stage One report and both packages;
  - a SHA256SUMS file.
- **Kept apart:** the R20.2 body, only if you approve and it is still available, with the token replaced by "[token removed]".
- **What it needs from you:** folder access for that one path. Nothing is written until you approve, and the application folder is not touched.

7. Execution order, result capture and stop rules

There would be 23 statement executions, strictly one at a time. Each response is saved before the next statement is sent, and the run stops at the first failure or missing result. **[Proposal]** Nothing here is authorised yet.

Order.

Step	Statement	Purpose
1	T1	target confirmation (version, port, schema, account)

Step	Statement	Purpose
2	H0	timeout settings and performance_schema
3	H1	counter baseline
4–5	M1, then H1	columns, types, defaults; counter check
6–7	M2, then H1	sizes, engines, indexes; counter check
8–9	G1c, then H1	first look at the difference; counter check
10–11	G2r, then H1	number-level reconciliation; counter check
12–13	G3, then H1	journal legs; counter check
14–15	G4, then H1	voucher lines; counter check
16–17	L1, then H1	aflag and Bank_Date spread; counter check
18–19	L2, then H1	aflag and Bank_Date on Stage One rows; counter check
20–21	E1, then H1	plan for R20.2r, without running it; then H1, recorded as the new baseline but not used as a test
22–23	R20.2r, then H1	only if E1 met every finding in section 2 and every earlier counter check passed

G1 (reference) is not run. A step is skipped only where a stop rule says so; nothing is reordered.

T1, unchanged from Stage One. It has no hint and reads no table.

```
SELECT @@version AS mysql_version, @@port AS port, DATABASE() AS
selected_schema, CURRENT_USER() AS signed_in_account;
```

Before step 1.

- You choose a time when the local app is idle and sign in to phpMyAdmin yourself.
- I take no part in the sign-in.
- No other tab sends queries during the run.

For every statement.

1. **Gate.** Compute the SHA-256 of the text about to be sent. It must equal the value in section 8, and the text must be one statement: SELECT, or EXPLAIN for E1 only.
2. **Send** through phpMyAdmin's own SQL handler with "show all rows", as in Stage One. Record the UTC start time to the millisecond.

3. **Wait** until the response has fully arrived. Nothing else is sent meanwhile. Record the finish time and the HTTP status.
4. **Save the raw response first**, before any parsing.
   - Replace phpMyAdmin's session token values with "[token removed]" and change nothing else.
   - Record the SHA-256 of the saved file.
5. **Read it.**
   - Parse the result table, and compare the row count with phpMyAdmin's "Showing rows … total".
   - Note phpMyAdmin's "Query took" time and any warning or message it shows.
   - Save the rows as a pipe-separated file with its SHA-256.
6. **Apply the checks** for that statement (below). Only then is the next statement sent.

Stop rules. On any of these the run stops and is reported. Nothing is retried, reworded or reordered.

- An error, including 3024 (time limit reached), or an HTTP status other than 200.
- A response with no result table, or one that could not be saved.
- A row count different from phpMyAdmin's total.
- A text whose SHA-256 differs from the approved value.
- A sign-in prompt, or any sign that a second statement is in flight.
- T1 differs from 8.4.7 / 3306 / avenir.
- H0 shows a nonzero max_execution_time, or performance_schema off.
- An H1 check fails (section 3), or a counter moves unexpectedly.
- G2r′s SUM(amount_sup0_authorised_rows) differs from 6,535,170.97.
- R20.2r′s monthly not_found_by_number differs from R20.1, or its "only or both" columns do not add up.
- E1 fails any finding in section 2. Then R20.2r does not run; this is the one step that is skipped.

Not stops, but reported separately:

- G2r rows with voucher_rows > 1 (shared numbers, section 1);
- a G2r cr_9750 total that does not reproduce R0′s 6,833,545.97;
- L1 or L2 values outside the expected pattern.

Afterwards. One report, with the files listed in a SHA256SUMS file. The files go to the evidence folder only if you have approved it (section 6).

8. Statement checksums and decisions needed

Thirteen statement texts are proposed, plus G1 kept as a reference. Each is approved by its SHA-256, computed over the exact text in its code block in this revision. **[Proposal]**

Id	Section	Runs	SHA-256
T1	3, 7 (unchanged from Stage One)	once	ea630a9d40b12b39a1ef0a113b2743e526356f09e859b81c4ded4baf1180e37d
H0	3	once	9a6387bf0ab009b402c2bf7b2b6562ce922cf471f5236be46bd357f9ef3d30a6
H1	3	11 times	b5d71484e3ae925ccbc930151bbd4f78ce9f7ed7ee191a1b00b33c7061f926da
M1	1	once	534af761d2de9c25ffff7e25ad5f1b51c2ab192b8bad36fd91b18ad8f68a05cf
M2	1	once	a0dcb5261c0c5569b9d2241587ed18dce955710609d44f592980b4fc0e3c73bd
G1	1 (reference)	**not run**	333b765bfc4253a2421c1dc2d901cc18dc598d13604dcfb6604ac713c4aa1e6c
G1c	1	once	c5b2027aae6a659d22bba174f1ca25670f19f8a9cb1ee3668f08a546802e9e33
G2r	1	once	af0f108157857706bccab98ba8561d18426691a991601fac8bd84a3e99d7b752
G3	1	once	095226d60b7d1898a30d96f82652124b688e997e18e5f5ffe9900bd98e5e39cd
G4	1	once	3f11a21874d16511f3f3a94c4ef348dd5d3ab3134c52af7231a6f6ac1c5a3c45
L1	4	once	d2a40f46fb36c160b5c8b57a5972a1f3fd6504162bf8ec8a755f70dbb7c33f42
L2	4	once	e886caac95659894049359fb707b07f8aae261876b62029edebf49f63d192dc7
E1	2	once, not executed as a query	3aae853b7b6e664e58e5fd565f628d9bca94a761c3c930452be77dd0e9d098f5

Id	Section	Runs	SHA-256
R20.2r	2	once, only if E1 and every counter check pass	f5d973b66f9271c48b35d0b925b304d61f98c06b08ed6579552ff1d0e4e40c3c

G2, R20.2r and E1 from the first package are superseded. Their old checksums are not to be used. M1, M2, G3, G4 and the texts now called L1 and L2 are unchanged from the first package. G3 still includes Bank_Date.

Decisions for you.

1. **Statement texts:** approve the thirteen by checksum, or name changes.
2. **Scope beyond `avenir`:** H1 reads performance_schema.global_status, which is server counters only. M1 and M2 read information_schema filtered to `avenir`. Both need your explicit agreement.
3. **E1 and R20.2r:** whether R20.2r may run, automatically, when E1 meets every finding in section 2. The alternative is to stop after E1 and ask you.
4. **Run window:** a time when the local app is idle.
5. **Containment (section 5):** refuse or guard for R1 and R2; the guards for R3–R6; the voucher-number lock for every voucher writer; and the further code reads F-1 to F-7.
6. **Evidence:** a folder outside C:\wamp64\www, with access granted, and whether to save the R20.2 body if it still exists.

Unchanged.

- No statement, plan check, test, build, installation, correction, data edit or permission change has been made.
- The target remains your localhost application and schema `avenir`. Live, avenir_rv, the cloud test database and other application schemas remain excluded.
- Ravi has not been contacted, and no records were sent.
- A0, D14 and containment implementation stay pending. Staff and access decisions stay with you.
- The Stage One report, the first follow-up package, A1, A2, r3.3–r3.5 and the Security proposal are unchanged.